

CMPT 276 - Assignment 4

Kaitlyn Lum and Marika Screpnek

During Assignment 4, we refactored several aspects of our code. We focused on refactoring our Board.java class, and in doing so improved the anti-pattern of Board.java being the “God Class” of the system. In this report, we have put the commit number within brackets near the end of the paragraph for each aspect we refactored.

The first occurrence of a bad code smell we encountered was a **large class** (Board.java), more specifically, too many instance variables. We had ArrayLists for Cones, Parked Cars, and Coins all within the board class. To refactor, we created a new class called EntityLists to combine related attribute variables and put them in a class of their own. We then moved the 3 arrays that held the Cones, Parked Cars, and Coins, to be attributes of EntityLists.java, replacing the 3 ArrayLists instances in Board.java with 1 instance of EntityLists.
(b020ddc, b21a06a, bcbf673)

The second occurrence of a bad smell we encountered was a **long method** (paintComponent()) in Board.java. We refactored paintComponent() by creating separate, smaller methods instead of one large method to increase readability and understandability (want to understand a method without having to add comments). We created 4 smaller, separate methods to draw entities such as Coins, Cones, Parked Cars, and Lost Notes, instead of drawing them using several for loops within the paintComponent() method.
Draw coins method: (c84e666)
Draw cones method: (d5ddc73)
Draw Parked Cars method: (5942ecf)
Draw Lost Note method: (c346371)

The third occurrence of a bad smell we encountered was **inappropriate intimacy** in the paintComponent() method in Board.java. paintComponent() was calling checkGameEnd(). This made paintComponent handle both rendering and game logic which violates the Single Responsibility Principle. To fix this we put the checkGameEnd() into actionPerformed() instead of inside the paintComponent() because it made more logical sense to be the responsibility of actionPerformed(). It should be checked whether the game has ended due to an action performed. We also changed the checkGameEnd() where another boolean was added to ensure that multiple end screens do not appear. The name of this new boolean is game_ended.
(a56dbe0, ad45d5b).

The fourth occurrence of a bad smell we encountered was **feature envy**. The Board class had a method called drawBarrier(), when Barrier was its own class. We moved the logic of the drawBarrier() to the Barrier class (as Barrier.draw()) instead of the board class so the barrier was responsible for drawing itself, and the Board class can focus on itself rather than other classes such as Barrier.
(e24738c).

The fifth occurrence of a bad smell we encountered was a **data clump**. Board.java contained instances for ROWS, COLUMNS, and CELL_SIZE that were often accessed together and therefore clumping as fields. To resolve this smell we created a new class called Dimensions.java. In doing this, we reduced the amount of instance variables for board

(ROWS, COLUMNS, CELL_SIZE → Dimensions), and removed the data clumping as now when accessing these fields we aren't constantly calling all 3 fields but now only call 1 field. We also implemented getters within the Dimensions class so that we could access the values of the private variables instead of using direct access to those fields (improved encapsulation).
(65677a0, ff73d8a, 515f75a)

We then encountered 3 occurrences of the **duplicate code** smell filling our sixth, seventh, and eighth bad code smells. These could also be considered as **speculative generality** smells as we were hard-coding entity coordinates with the thought that we could read from a .json file or put it into fields later, making us create unnecessarily large populating methods: We used hard coded coordinates in the body of methods for creating Coins (sixth occurrence), Cones (seventh occurrence), and Parked Cars (eighth occurrence). To resolve this issue, we created 3 2D ArrayLists holding the x and y positions of where the entities needed to be (coin_coords, cone_coords, and pc_coords). With these variables, we were able to complete creation of these objects using for loops instead of multiple repetitive lines of code with only minor changes to the hard coded position in each line. (b020ddc, 5c70f07, 5f9b465, 37cbffd, ef4d583, 99be73f)

The tenth and final occurrence of a bad smell we encountered through doing the above refactoring was once again a **large class** (Board.java) with too many instance variables. After creating the 3 different coord ArrayLists as instances in Board.java, we noticed that our attribute list was getting large again. To resolve this, we created an EntityCoordinate.java class which held each type of entity's coords ArrayList as fields. In the Board class, we removed these instances, added an EntityCoordinate instance, and updated the creation of Entities onto the Board using the EntityCoordinate instance.
(b020ddc, 5c70f07, 5f9b465, 37cbffd, ef4d583, 99be73f)